AUTH SYSTEM

Complete Developer Documentation

DOCUMENT 2 OF 4

Environment Setup Guide — From Zero to Running

1. Software You Must Install

Install the following software in the exact order listed. Each item includes a verification command — run it in your terminal to confirm the installation worked.

What is a Terminal?

The Terminal (Mac/Linux) or Command Prompt (Windows) is a text interface where you type commands to control your computer. In PyCharm it is the 'Terminal' tab at the bottom. In VS Code it is View > Terminal. You will use this constantly as a developer.

1.1 Installation Order — Windows

#	Software	Download From	Verify Command
1	Python 3.12	python.org/downloads — check Add Python to PATH	python --version
2	Git	git-scm.com — choose Git from command line option	git --version
3	Node.js 20 LTS	nodejs.org — download the LTS version	node --version
4	Docker Desktop	docker.com/products/docker-desktop	docker --version
5	PyCharm Community	jetbrains.com/pycharm — Community is free	Open the IDE
6	VS Code	code.visualstudio.com	code --version

1.2 Installation Order — Mac

Open the Terminal application (Cmd + Space, type Terminal, press Enter) and run each command:

```
# Step 1 — Install Homebrew (Mac package manager)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Step 2 — Install Python 3.12
brew install python@3.12

# Step 3 — Install Git
brew install git

# Step 4 — Install Node.js 20
brew install node@20

# Step 5 — Install Docker Desktop
# Download from docker.com/products/docker-desktop (graphical installer)

# Step 6 — Verify all installations
python3 --version   # Should show Python 3.12.x
git --version       # Should show git version x.x.x
node --version      # Should show v20.x.x
docker --version    # Should show Docker version x.x.x
```

1.3 VS Code Extensions to Install

Open VS Code, press Ctrl+Shift+X (Windows) or Cmd+Shift+X (Mac) to open Extensions, and search for and install each:

- ESLint — catches JavaScript and TypeScript errors as you type
- Prettier — automatically formats your code on save
- Tailwind CSS IntelliSense — autocompletes Tailwind class names
- TypeScript and JavaScript Language Features — already installed by default
- GitLens — shows who changed each line of code and when

2. Setting Up the Backend (FastAPI)

2.1 Open Project in PyCharm

1. Open PyCharm
2. Click Open (not New Project)
3. Navigate to your auth-system/backend-api folder
4. Click Open — click Trust Project when asked
5. Wait for PyCharm to index the files (progress bar at bottom)

2.2 Create a Virtual Environment

A virtual environment is a private folder that contains Python packages specifically for this project. It prevents conflicts between different projects on your computer.

6. In PyCharm: File > Settings (Windows) or PyCharm > Preferences (Mac)
7. Navigate to: Project: backend-api > Python Interpreter
8. Click the gear icon > Add Interpreter > Add Local Interpreter
9. Choose Virtualenv Environment > New environment
10. Location should auto-fill to backend-api/venv — keep it
11. Base interpreter: select Python 3.12 from the dropdown
12. Click OK

What just happened?
PyCharm created a venv/ folder inside backend-api. This folder contains a private copy of Python and will store all project packages. The (venv) prefix in your terminal confirms it is active. This folder is in .gitignore and will never be committed to Git.

2.3 Install Python Packages

In the PyCharm terminal (Terminal tab at the bottom), run:

```
# You should see (venv) at the start of the line
# This confirms your virtual environment is active

# Install all required packages
pip install -r requirements.txt

# This will take 2-4 minutes — completely normal
# You will see many lines of output as packages download
```

```
# Verify key packages installed correctly
pip show fastapi
pip show sqlalchemy
pip show redis
```

2.4 Set Up PostgreSQL Database

PostgreSQL must be running before the backend can start. Use Docker to run it — much easier than a manual installation:

```
# Run PostgreSQL in Docker (from any terminal)
docker run --name auth-postgres \
  -e POSTGRES_DB=authdb \
  -e POSTGRES_USER=authuser \
  -e POSTGRES_PASSWORD=devpassword \
  -p 5432:5432 \
  -d postgres:16-alpine

# Verify it is running
docker ps
# You should see auth-postgres in the list

# Test the connection
docker exec -it auth-postgres psql -U authuser -d authdb -c 'SELECT version();'
```

2.5 Set Up Redis

```
# Run Redis in Docker
docker run --name auth-redis \
  -p 6379:6379 \
  -d redis:7-alpine

# Verify Redis is running
docker exec -it auth-redis redis-cli ping
# You should see: PONG
```

2.6 Create Your .env File

The .env file stores all your local configuration. It NEVER goes to GitHub. Create it from the template:

```
# In PyCharm terminal, inside backend-api/
cp .env.example .env
```

```
# Generate your SECRET_KEY (run this, copy the output)
python -c "import secrets; print(secrets.token_urlsafe(64))"

# Generate your OTP_HMAC_KEY
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

Now open the .env file in PyCharm and set these minimum values to get started:

```
APP_ENV=development
DEBUG=true
SECRET_KEY=paste-your-generated-key-here
OTP_HMAC_KEY=paste-your-generated-key-here
DATABASE_URL=postgresql+asyncpg://authuser:devpassword@localhost:5432/authdb
REDIS_URL=redis://localhost:6379/0
COOKIE_SECURE=false
ALLOWED_ORIGINS=["http://localhost:3000"]
ALLOWED_HOSTS=["localhost","127.0.0.1"]
ERROR_NOTIFICATIONS_ENABLED=false

# Leave these as fake values for now
# You do not need real SMS/email for basic testing
TWILIO_ACCOUNT_SID=fake_for_now
TWILIO_AUTH_TOKEN=fake_for_now
TWILIO_FROM_NUMBER=+15005550006
SENDGRID_API_KEY=fake_for_now
EMAIL_FROM=test@localhost.com
```

2.7 Run Database Migrations

Migrations create the database tables. You must run this before starting the server for the first time:

```
# In PyCharm terminal, inside backend-api/
alembic upgrade head

# Expected output:
# INFO  [alembic.runtime.migration] Running upgrade -> 0001, Initial schema

# Verify tables were created
docker exec -it auth-postgres psql -U authuser -d authdb -c "\dt"
# You should see: users, user_sessions, audit_logs
```

2.8 Start the Backend Server

```
# In PyCharm terminal, inside backend-api/
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

```
# Expected output:
# INFO:     Started server process
# INFO:     Waiting for application startup.
# INFO:     redis_pool_ready
# INFO:     app_started env=development version=1.0.0
# INFO:     Application startup complete.
# INFO:     Uvicorn running on http://0.0.0.0:8000
```

Open your browser and go to http://localhost:8000/docs to see the automatic API documentation. This only works in development mode.

3. Setting Up the Frontend (Next.js)

3.1 Open Frontend in VS Code

13. Open VS Code
14. File > Open Folder
15. Navigate to auth-system/frontend-app
16. Click Open

3.2 Install Node.js Packages

Open VS Code terminal (View > Terminal or Ctrl+`) and run:

```
# Inside frontend-app/
npm install

# This reads package.json and downloads all dependencies
# Creates a node_modules/ folder — takes 1-3 minutes
# node_modules/ is in .gitignore and will NOT go to GitHub

# Verify Next.js is installed
npx next --version
```

3.3 Create Frontend Environment File

Next.js uses a different naming convention for environment files:

```
# Inside frontend-app/ — create this file
# File name: .env.local

# Content to put in .env.local:
NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1
INTERNAL_API_URL=http://localhost:8000/api/v1
NODE_ENV=development
```

NEXT_PUBLIC_ Prefix

In Next.js, only variables starting with NEXT_PUBLIC_ are available in the browser. Variables WITHOUT this prefix are only available on the server. Never put secret keys in NEXT_PUBLIC_ variables — they will be visible to everyone who views your page source.

3.4 Start the Frontend Server

```
# In VS Code terminal, inside frontend-app/
npm run dev

# Expected output:
# > frontend-app@1.0.0 dev
# > next dev
# - ready started server on 0.0.0.0:3000
# - info: automatically enabled Fast Refresh
```

Open your browser and go to http://localhost:3000 to see the login page.

4. Easier Option — Run Everything with Docker

Instead of starting each service separately, Docker Compose starts everything with one command. This is the recommended approach once you are comfortable with the project.

4.1 The Easy Start Command

```
# Navigate to the project root (auth-system/)
cd auth-system

# Copy the backend environment file first
cp backend-api/.env.example backend-api/.env
# Then edit backend-api/.env with your values

# Start ALL services: postgres, redis, backend, frontend, nginx
docker-compose up --build

# First time takes 5-10 minutes to download images
# Subsequent starts take 10-30 seconds

# In a NEW terminal tab, run migrations
docker-compose exec backend alembic upgrade head

# Access the application
# Frontend:   http://localhost:3000
# Backend API: http://localhost:8000
# API Docs:   http://localhost:8000/docs
```

4.2 Useful Docker Commands

Command	What It Does
docker-compose up	Start all services (shows logs)

Command	What It Does
docker-compose up -d	Start all services in background
docker-compose down	Stop all services
docker-compose down -v	Stop and delete all data (fresh start)
docker-compose logs backend	View backend logs only
docker-compose logs -f backend	Follow backend logs in real time
docker-compose restart backend	Restart only the backend service
docker-compose exec backend bash	Open terminal inside backend container
docker ps	List all running containers
docker-compose pull	Download latest images

5. Verifying Everything Works

Run through this checklist after setup to confirm all services are healthy:

	What to Check	Expected Result
1	Open http://localhost:3000	Login page loads with all four login method tabs
2	Open http://localhost:8000/health	Browser shows: {"status":"ok","version":"1.0.0"}
3	Open http://localhost:8000/docs	Swagger UI loads showing all API endpoints
4	Open http://localhost:8000/readyz	Shows {"status":"ok","checks":{"redis":"ok","db":"ok"}}
5	Register a test account at /register	Form submits, success message appears
6	Check PyCharm terminal	No ERROR lines in the log output

5.1 Common Problems and Solutions

Problem	Solution
Port 8000 already in use	Another process is using the port. Run: lsof -i :8000 (Mac) or netstat -ano | findstr :8000 (Windows) to find it, then stop it
Cannot connect to PostgreSQL	Confirm Docker container is running: docker ps. Restart it: docker start auth-postgres
ModuleNotFoundError	Your virtual environment is not active. In PyCharm terminal you should see (venv) at the start
CORS error in browser	Check ALLOWED_ORIGINS in .env includes http://localhost:3000 exactly, with the correct port
422 Unprocessable Entity	The data you sent does not match the expected format. Read the error detail in the response body
npm install fails	Delete node_modules/ folder and package-lock.json, then run npm install again
alembic command not found	Virtual environment is not active. Run: source venv/bin/activate (Mac) or venv\Scripts\activate (Windows)

6. Creating Your First Admin User

After setup, create the first administrator account using the included script. This bypasses the normal registration flow and directly creates a verified superuser:

```
# In PyCharm terminal, inside backend-api/
python scripts/create_superuser.py \
  --email admin@yourdomain.com \
  --username admin \
--password "YourStrongPass@123" \
  --full-name "Admin User" \
  --phone "+919876543210"

# Expected output:
# Superuser created: admin@yourdomain.com

# Now try logging in at http://localhost:3000/login
# Use the password login tab
# Enter: admin@yourdomain.com and your password
```

7. Running the Test Suite

Tests verify that every part of the application works correctly. Run them after any change to confirm nothing is broken:

```
# In PyCharm terminal, inside backend-api/

# Install test dependencies (first time only)
pip install aiosqlite pytest-asyncio httpx

# Run all tests
pytest -v

# Run with coverage report (shows which lines are tested)
pytest -v --cov=app --cov-report=term-missing

# Run only a specific test file
pytest tests/test_login.py -v

# Run only tests matching a name pattern
pytest -v -k "otp"

# Expected final line:
# X passed in X.XXs
```

Why Tests Matter

Tests are automated checks that run your code and verify the output is correct. Without tests, every change requires manually clicking through the application to check nothing broke. With tests, you get that assurance in seconds. Always run tests before committing code.